

PRD: Smart Energy Consumption Monitoring & Recommendation System

Sep 25, 2026 · @Hem

1. Executive Summary

We will build a web platform that turns raw household electricity data into appliance-level insight, 7–30 day forecasts, anomaly alerts, and ranked energy-saving recommendations with quantified kWh, cost and CO2 impact. A digital twin lets users test "what-if" changes before acting, and a tracking engine measures whether accepted recommendations actually reduced consumption.

Target outcome for the project demo: a household using the system identifies its top 3 energy-wasting appliances within 5 minutes, sees a forecast with $MAPE \leq 15\%$, and receives recommendations projected to save 10–20% of monthly consumption.

Item	Decision
Primary dataset	Kaggle Smart Home Energy Consumption Dataset (appliance-level, weather, occupancy)
Architecture	Modular monolith (FastAPI) + separate ML worker; React/TypeScript frontend; PostgreSQL
Core ML	XGBoost forecasting (Prophet baseline), Isolation Forest anomalies, Random Forest/XGBoost classification, hybrid rule + content-based recommender
Differentiators	Closed-loop recommendation tracking, digital twin simulator, SHAP-based explainability
Timeline	16 weeks across 6 phases
Deployment	Docker Compose on AWS EC2 (or Azure App Service), RDS PostgreSQL

2. Problem Statement

Households waste an estimated 10–30% of electricity because monthly bills show one aggregate number and no guidance. Users cannot see which appliance drives cost, when peaks happen, or whether changes they made worked.

Gap today	Consequence	How the system closes it
Aggregate monthly bill only	No appliance attribution	Appliance-wise breakdown and ranking

No time-of-day visibility	Peak-tariff usage goes unnoticed	Peak hour detection and heatmaps
No forward view	Bill shock at month end	7/30-day forecasts with confidence bands
No abnormal-usage signal	Faulty or left-on devices run for days	Real-time anomaly detection and alerts
Generic tips ("turn off lights")	Low adoption, unknown value	Personalized, quantified recommendations
No feedback loop	Users don't know if actions worked	Before/after tracking and effectiveness scores
Risky to experiment	Users hesitate to change routines	Digital twin what-if simulation

3. Business & System Objectives

The system succeeds if it measurably changes household behavior, not just displays data. Business objectives map to measurable targets; system objectives map to modules.

#	Business objective	Target (demo / pilot)
B1	Reduce household energy consumption	10–20% projected monthly kWh reduction per household
B2	Reduce electricity cost	Show ₹/\$ savings per recommendation; ≥ ₹300 (~\$4)/month for a typical 300 kWh home
B3	Reduce carbon footprint	Report kg CO ₂ avoided using grid emission factor (default 0.71 kg/kWh, India CEA; configurable)
B4	Drive engagement	≥ 40% recommendation acceptance rate; ≥ 3 dashboard sessions/week
B5	Demonstrate end-to-end AI/ML	4 ML capabilities in production path with documented evaluation

#	System objective	Module(s)
O1	Ingest data via CSV, manual entry, simulator	Data Ingestion
O2	Store and manage historical records	Data Storage

O3	Interactive dashboards	Dashboard & Visualization
O4	Detect normal/high patterns, anomalies, peaks	Analytics Engine, ML Layer
O5	Forecast consumption	ML Layer (Forecasting)
O6	Personalized recommendations	Recommendation Engine
O7	Estimate energy, cost, CO2 savings	Savings Estimation Engine
O8	Track recommendation effectiveness	Recommendation Tracking Engine
O9	What-if simulation	Digital Twin
O10	Visible AI/ML throughout	All, with explainability layer

Out of scope (v1): live smart-meter/IoT hardware integration, device control (switching appliances), multi-tenant utility portal, payments.

4. User Personas

Three primary personas drive v1 design; secondary personas get read-only analytics and export.

Persona	Profile	Goals	Pain points	Key features used
Priya, Homeowner (primary)	38, family of 4, 3BHK, ~450 kWh/month, 2 ACs, geyser, EV charger planned	Cut bill 15%, understand AC cost	Bill spikes in summer, no idea why	Appliance analysis, forecasts, recommendations, digital twin
Arjun, Apartment Resident (primary)	29, working professional, 1BHK, ~180 kWh/month, away 10 h/day	Avoid waste while away, simple alerts	Devices left on, standby drain	Anomaly alerts, peak- hour insights, behavior tips
Meera, Student (primary)	21, shared hostel/PG, split bill with 3 roommates	Fair split, low- effort savings	No per- person visibility, low budget	Manual entry, daily view, gamified improvement score
Energy Auditor (secondary)	Consultant auditing homes	Fast diagnosis,	Manual logging	CSV upload, anomaly report, PDF/CSV export

		exportable reports		
Utility Analyst (secondary)	Demand-side management team	Aggregate peak-shift potential	No appliance-level data	Aggregated anonymized segments, forecast export
Sustainability Researcher (secondary)	Academic	Behavioral datasets, model benchmarks	Data access	Anonymized export, model metrics

Roles (RBAC): user (own household), auditor (assigned households, read + export), admin (users, tariffs, emission factors, model retraining).

5. Dataset & Data Preprocessing

The pipeline is built around a canonical internal schema so any Kaggle variant, user CSV or simulator output maps into the same tables. Column names below are from memory of the common Kaggle smart-home datasets and are approximate; confirm against the downloaded file in Week 1.

5.1 Expected source features

Feature group	Example columns	Type	Use
Time	time (Unix epoch, minute-level)	datetime	Index, calendar features
Aggregate load	use [kW], House overall [kW]	float (kW)	Target for forecasting
Generation	gen [kW], Solar [kW]	float (kW)	Net load, solar offset
Appliance load	Dishwasher, Fridge, Microwave, Furnace 1/2, Home office, Living room, Kitchen 12/14/38, Garage door, Well, Barn, Wine cellar [kW]	float (kW)	Appliance analysis, disaggregated forecasts
Weather	temperature, apparentTemperature, humidity, pressure, windSpeed,	float	Exogenous regressors

	cloudCover, precipIntensity, dewPoint, visibility		
Weather category	summary, icon	categorical	One-hot / embedding
Occupancy (derived or given)	occupancy flag, household size	int/bool	Behavior features

5.2 Canonical schema (after mapping)

household_id, appliance_id, ts (UTC, 15-min bucket), energy_kwh, power_kw_avg, temperature_c, humidity_pct, weather_code, occupancy, source

5.3 Preprocessing pipeline

1. **Parsing & typing:** convert epoch to UTC datetime; fix the known malformed trailing row; coerce numerics; convert °F to °C if needed; kW → kWh via kW × interval_hours.
2. **Deduplication:** drop duplicate timestamps per household/appliance, keeping the last.
3. **Missing values:**
 - Gaps ≤ 30 min: linear/time interpolation.
 - Gaps 30 min–6 h: fill with same-hour value from the previous week (seasonal naive).
 - Gaps > 6 h: leave NaN and flag is_imputed = false, gap = true; exclude from training windows.
 - Categorical weather: forward-fill; cloudCover string artifacts → NaN → forward-fill.
4. **Outlier treatment:**
 - Physical bounds: negative loads → 0; appliance kW > rated max × 1.5 → cap.
 - Statistical: rolling 7-day median ± 5 × MAD flagged as outlier_flag.
 - Outliers are **flagged, not deleted**, so the anomaly model can learn from them; forecasting uses winsorized values (1st–99th percentile).

5. **Time-series aggregation:** resample minute data to 15-min (storage grain), hourly (analytics, anomaly), daily (forecast, classification), weekly/monthly (dashboards).
6. **Feature engineering:**
 - Calendar: hour, day-of-week, weekend flag, month, season, holiday flag.
 - Cyclical encodings: sin/cos of hour and day-of-year.
 - Lags: t-1, t-24, t-168 (hourly); t-1, t-7 (daily).
 - Rolling stats: 24 h / 7 d mean, std, max.
 - Weather: cooling degree hours $\max(0, T - 24^{\circ}\text{C})$, heating degree hours $\max(0, 18^{\circ}\text{C} - T)$, temperature $\times$ hour interaction.
 - Appliance: share of total, on/off state ($\text{kW} > \text{threshold}$), run duration, cycles/day, standby kWh.
 - Behavior: peak-hour share (18:00–22:00), night base load (01:00–05:00 min), load factor = mean/peak.
7. **Normalization:** StandardScaler for SVM/autoencoder/LSTM; MinMaxScaler [0,1] for LSTM inputs; none for tree models. Scalers fit on training split only and persisted with the model artifact.
8. **Splitting:** chronological split 70/15/15 (train/validation/test); walk-forward cross-validation with 5 folds for model selection. No random shuffling.
9. **Label generation (classification):** per household-day, label = Low / Normal / High by percentile of daily kWh vs. that household's 30-day baseline ($< P25$, $P25-P75$, $> P75$) plus an optional global K-Means segment label for comparison.

Data quality report (generated per upload): row count, date range, % missing per column, % imputed, outliers flagged, duplicate count.

6. Functional Requirements

Requirements are numbered FR-<module>-<n> and prioritized MoSCoW (M = Must, S = Should, C = Could). All Must items are required for the final demo.

6.1 Data Ingestion Layer (ING)

ID	Requirement	Priority
FR-ING-1	User uploads CSV/XLSX up to 100 MB; system auto-detects delimiter, timestamp column and units	M

FR- ING-2	Column-mapping wizard maps source columns to canonical schema; mappings saved as reusable templates (Kaggle template pre-loaded)	M
FR- ING-3	Large files processed asynchronously; upload returns job_id; UI shows progress and final data quality report	M
FR- ING-4	Manual entry form: date, appliance, hours used or kWh, optional meter reading; bulk grid entry for a week	M
FR- ING-5	Meter-reading mode: user enters cumulative meter readings; system derives interval consumption	S
FR- ING-6	Validation rules: timestamp parseable, not in future, non-negative kWh, appliance kWh $\leq$ rated_power $\times$ hours $\times$ 1.2, total $\geq$ sum(appliances) $\times$ 0.9	M
FR- ING-7	Invalid rows rejected into a downloadable error file with row number and reason; valid rows committed	M
FR- ING-8	Smart Home Simulator generates synthetic data for N days from a household profile (appliances, schedules, weather) at 1/15/60-min resolution	M
FR- ING-9	Simulator can stream data (1 record every X seconds) to emulate live meter feed and trigger real-time anomaly checks	S
FR- ING-10	Simulator supports fault injection (e.g., fridge compressor stuck on, AC left on overnight) for anomaly demo	S
FR- ING-11	Every record stores source (csv / manual / simulator) and ingestion_job_id for lineage	M
FR- ING-12	Idempotent upload: re-uploading the same file (SHA-256 hash) does not duplicate rows	S

6.2 Data Storage Layer (STO)

ID	Requirement	Priority
FR- STO-1	User registration, login (email + password, bcrypt), JWT access (15 min) + refresh (7 days) tokens	M
FR- STO-2	Household profile: size, type, location, tariff plan, appliances inventory	M

FR- STO-3	Consumption stored at 15-min grain; hourly/daily materialized views refreshed after each ingestion	M
FR- STO-4	Recommendation history with status lifecycle (generated → viewed → accepted/rejected → completed/expired)	M
FR- STO-5	Forecast history: every forecast run stored with model version, horizon, and predicted vs. actual once actuals arrive	M
FR- STO-6	Savings reports stored monthly per household	M
FR- STO-7	Digital twin scenarios saved with inputs and outputs	M
FR- STO-8	Model registry table: model name, version, metrics, artifact path, trained_at, is_active	S
FR- STO-9	Soft delete + GDPR-style hard delete of all household data on request	S
FR- STO-10	Nightly automated backup, 7-day retention	S

6.3 Analytics Engine (ANA)

ID	Requirement	Priority
FR- ANA-1	Compute daily/weekly/monthly totals, averages, % change vs. previous period and vs. same period last year	M
FR- ANA-2	Peak hour detection: hours whose mean load exceeds P90 of household hourly load; report top 3 peak windows and their share of kWh	M
FR- ANA-3	Hour × weekday heatmap of average consumption	M
FR- ANA-4	Appliance-wise analysis: kWh, cost, share %, run hours, cycles, standby kWh, rank; efficiency index vs. benchmark kWh for that appliance type	M
FR- ANA-5	Base load (always-on) estimation = P5 of night-time load	M

FR- ANA-6	Weather sensitivity: regression of daily kWh on cooling/heating degree days; report kWh per °C	S
FR- ANA-7	Consumption segmentation: K-Means (k=4) on daily load profiles → segments e.g. Morning-peak, Evening-peak, Flat, Night-heavy	S
FR- ANA-8	Time-of-use cost analysis using tariff slabs and peak/off-peak rates	M
FR- ANA-9	Peer comparison against similar households (same size and type) from the dataset	C

6.4 Machine Learning Layer (ML)

ID	Requirement	Priority
FR- ML-1	Forecast household total daily kWh for 7 and 30 days, and hourly kWh for next 48 h, with 80%/95% prediction intervals	M
FR- ML-2	Forecast per-appliance daily kWh for top 5 appliances	S
FR- ML-3	Anomaly detection on hourly appliance and total load; each anomaly has score (0–1), severity (low/med/high), affected appliance, expected vs. actual kWh	M
FR- ML-4	Real-time anomaly scoring for streamed simulator data within 5 s of arrival	S
FR- ML-5	Classify each household-day as Low / Normal / High consumption; classify household into efficiency tier (Efficient / Average / Heavy)	M
FR- ML-6	SHAP explanations for forecasts and classifications; top 3 drivers shown in UI in plain language	M
FR- ML-7	Model training pipeline runnable via CLI and admin API; scheduled weekly retraining; new model promoted only if validation MAPE improves $\geq 2\%$	S
FR- ML-8	Cold start: households with < 14 days of data use a global model trained on the Kaggle dataset, scaled by household size	M
FR- ML-9	User feedback on anomalies ("expected" / "real issue") stored as labels for future supervised tuning	S

6.5 Recommendation Engine (REC)

ID	Requirement	Priority
FR- REC-1	Rule-based recommendations from a configurable rules table (condition → action template → savings formula)	M
FR- REC-2	ML-driven recommendations triggered by anomalies, forecast overrun vs. budget, and classification = High	M
FR- REC-3	Appliance-specific recommendations (e.g., AC setpoint +2°C, geyser timer, fridge replacement when efficiency index < 0.7)	M
FR- REC-4	Behavior-based recommendations (shift dishwasher/washing machine to off-peak, reduce standby)	M
FR- REC-5	Each recommendation shows: title, reason (data evidence), action steps, estimated kWh/cost/CO2 savings, effort (low/med/high), confidence	M
FR- REC-6	Ranking score = $0.5 \times \text{normalized savings} + 0.2 \times (1 - \text{effort}) + 0.2 \times$ confidence + $0.1 \times \text{historical acceptance of similar recs}$	M
FR- REC-7	Deduplicate: no repeat of a rejected recommendation for 30 days; max 5 active recommendations	M
FR- REC-8	Weekly recommendation digest (in-app; email optional)	C
FR- REC-9	"Simulate this" button opens the Digital Twin pre-filled with the recommendation	S

6.6 Savings Estimation Engine (SAV)

ID	Requirement	Priority
FR- SAV-1	kWh reduction = baseline appliance kWh – projected kWh after action (per day, month, year)	M
FR- SAV-2	Cost savings using household tariff (flat, slab, or time-of-use)	M
FR- SAV-3	CO2 reduction = kWh saved × grid emission factor (configurable per region)	M
FR- SAV-4	Savings shown as range (low/expected/high) reflecting forecast uncertainty	S

FR-SAV-5	Aggregate: cumulative realized vs. projected savings	M
FR-SAV-6	Equivalents for engagement (trees planted, km not driven)	C

6.7 Recommendation Tracking Engine (TRK)

ID	Requirement	Priority
FR-TRK-1	Record accept/reject/snooze with timestamp and optional reason	M
FR-TRK-2	On acceptance, snapshot baseline = 14-day pre-acceptance consumption of target appliance (weather-normalized)	M
FR-TRK-3	Evaluation window of 14 days post-acceptance; compare actual vs. counterfactual forecast (what would have happened)	M
FR-TRK-4	Improvement score per recommendation = $(\text{baseline} - \text{actual}) / \text{baseline} \times 100$, weather-adjusted	M
FR-TRK-5	Effectiveness score = realized savings / projected savings (capped 0–150%); labels: Effective $\geq 70\%$, Partial 30–70%, Ineffective $< 30\%$	M
FR-TRK-6	Household improvement score (0–100) = weighted mix of kWh trend, acceptance rate and effectiveness	S
FR-TRK-7	Effectiveness feeds back into recommendation ranking (FR-REC-6)	S
FR-TRK-8	Nudge if an accepted recommendation shows no change after 7 days	C

6.8 Digital Twin Module (DT)

ID	Requirement	Priority
FR-DT-1	Auto-build virtual household from appliance inventory + calibrated usage patterns learned from history	M
FR-DT-2	Each virtual appliance has rated power, duty cycle, daily hours, schedule, efficiency rating, standby power	M
FR-DT-3	Calibrate twin so simulated monthly kWh is within $\pm 10\%$ of actual last 30 days	M

FR- DT-4	What-if levers: change hours/day, cycles/day, setpoint, schedule shift, replace with efficient model, add/remove appliance, add solar	M
FR- DT-5	Output: projected monthly kWh, cost, CO2, peak demand, delta vs. current, 30-day projected curve	M
FR- DT-6	Compare up to 3 scenarios side-by-side	S
FR- DT-7	Save scenario; convert scenario to a tracked recommendation	S
FR- DT-8	Weather-aware: AC/heater consumption scales with forecast temperature	S

6.9 Dashboard & Visualization Module (DSH)

ID	Requirement	Priority
FR-DSH-1	Ten dashboards as specified in Section 12	M
FR-DSH-2	Global filters: household, date range, granularity, appliance(s)	M
FR-DSH-3	All charts interactive (hover tooltips, zoom on time series, legend toggles)	M
FR-DSH-4	Export chart/table as PNG/CSV; monthly report as PDF	S
FR-DSH-5	Responsive layout (desktop $\geq$ 1280 px, tablet, mobile 375 px)	M
FR-DSH-6	Light/dark theme	C
FR-DSH-7	In-app notifications for anomalies and new recommendations	M

7. Non-Functional Requirements

Targets are sized for a demo/pilot of up to 1,000 households and 50 concurrent users, with a clear path to 100k households.

ID	Category	Requirement	Measure / target
----	----------	-------------	------------------

NFR-1	Performance	Dashboard API responses	p95 < 500 ms for 1 year of 15-min data (~35k rows/household)
NFR-2	Performance	Initial page load	< 3 s on 10 Mbps; JS bundle < 500 KB gzipped
NFR-3	Performance	Forecast inference	< 2 s per household (30-day horizon)
NFR-4	Performance	CSV ingestion	500k rows processed < 60 s
NFR-5	Scalability	Stateless API containers behind load balancer; horizontal scaling	Scale 1 → N replicas without code change
NFR-6	Scalability	Heavy jobs (training, bulk ingest, batch forecasts) on async worker queue (Celery + Redis)	API never blocks > 1 s on ML work
NFR-7	Scalability	Consumption table partitioned by month; TimescaleDB extension optional	Query time flat as data grows
NFR-8	Reliability	Uptime (pilot)	99.5% monthly
NFR-9	Reliability	Failed jobs retried 3× with exponential backoff; dead-letter logged	0 silent failures
NFR-10	Reliability	Fallback: if ML model unavailable, show seasonal-naive forecast and rule-based recommendations	Graceful degradation
NFR-11	Security	HTTPS/TLS 1.2+, JWT auth, bcrypt (cost 12), RBAC on every endpoint	OWASP Top 10 addressed
NFR-12	Security	Input validation via Pydantic; parameterized SQL via SQLAlchemy; file-type and size checks on upload	No injection findings in scan
NFR-13	Security	Secrets in environment / AWS Secrets Manager, never in repo; rate limit 100 req/min/user	—

NFR-14	Maintainability	Layered code (routers / services / repositories), type hints, mypy + ESLint + Prettier, pre-commit hooks	≥ 70% backend test coverage
NFR-15	Maintainability	OpenAPI docs auto-generated; ADRs for key decisions; CI on every PR (GitHub Actions)	—
NFR-16	Explainability	Every forecast, anomaly, classification and recommendation shows "why" (SHAP top features, rule triggered, evidence chart)	100% of ML outputs explained
NFR-17	Explainability	Model cards per model: data, metrics, limitations	Published in repo
NFR-18	Data privacy	Household data isolated by household_id; row-level checks in repository layer	No cross-tenant access
NFR-19	Data privacy	Encryption at rest (RDS/AES-256) and in transit; PII (email, name) separated from consumption data	—
NFR-20	Data privacy	Consent at signup; data export and deletion on request; research exports anonymized (k-anonymity ≥ 5)	GDPR / India DPDP Act 2023 aligned
NFR-21	Usability	WCAG 2.1 AA contrast, keyboard navigation; key insight visible without scrolling	SUS score ≥ 70 in user test
NFR-22	Observability	Structured JSON logs, Prometheus metrics, model drift monitor (weekly MAPE on actuals)	Alert if MAPE > 20%

8. AI/ML Architecture

Recommended stack for a final-year project: XGBoost as the primary forecaster with Prophet as the explainable baseline, Isolation Forest for anomalies (autoencoder as the advanced variant), Random Forest for classification (XGBoost compared), and a hybrid rule-based + content-based recommender. This mix is accurate, trains on a laptop in minutes, is explainable with SHAP, and still shows deep learning (LSTM, autoencoder) as benchmarked alternatives.

flowchart LR

A[Raw data
CSV / manual / simulator] --> B[Preprocess &
feature store]

B --> C[Forecasting
XGBoost + Prophet]

B --> D[Anomaly detection
Isolation Forest]

B --> E[Classification
Random Forest]

C --> F[Recommendation
engine]

D --> F

E --> F

F --> G[Savings
estimator]

G --> H[Tracking &
feedback]

H --> F

B --> I[Digital twin
calibration]

I --> G

Features flow into three model families whose outputs trigger and rank recommendations; tracking results feed back into ranking.

8.1 Forecasting models

Model	How it's used	Advantages	Disadvantages	Expected MAPE (daily)
ARIMA / SARIMAX	Statistical baseline; SARIMAX with temperature as exogenous	Well understood, confidence intervals, good for viva theory	Needs stationarity tuning; one model per series; weak with multiple seasonalities; slow on minute data	15–25%
Prophet	Baseline and explainable decomposition (trend, weekly, yearly, holidays)	Handles missing data, holidays, multiple seasonality; decomposition charts great for UI	Less accurate on short-horizon hourly data; limited non-linear weather effects	12–20%

XGBoost (recommended primary)	Regression on lag, rolling, calendar and weather features; recursive/direct multi-step	Highest accuracy on tabular features; fast; SHAP explainability; handles non-linearity	Needs feature engineering; no native uncertainty (use quantile regression or conformal intervals)	8–15%
LSTM	Sequence model on 168-h windows; advanced benchmark	Captures complex temporal dependencies; multivariate	Needs more data and GPU time; hard to explain; overfits small households	10–18%

Expected MAPE ranges are rough estimates to be validated in Phase 3.

Forecast design:

- Targets: hourly total kWh (48 h), daily total kWh (7 and 30 days), daily per-appliance kWh (top 5).
- XGBoost: n_estimators 500–1500, max_depth 4–8, learning_rate 0.03–0.1, tuned by Optuna with walk-forward CV. Prediction intervals from quantile models ($\alpha = 0.1, 0.9$) or split conformal.
- LSTM: 2 layers $\times$ 64 units, dropout 0.2, 168-step input, Adam, early stopping on validation loss.
- Ensemble option: weighted average of XGBoost and Prophet (weights by inverse validation MAPE).
- Model selection is automatic per household: best validation MAPE wins; global model for cold start.

8.2 Anomaly detection

Model	Input	Advantages	Disadvantages	Role
Isolation Forest (recommended)	Hourly features: kWh, deviation from forecast, hour, weekday, temperature, appliance on-duration	Unsupervised, fast, scales well, few hyperparameters, works on mixed features	Contamination rate must be set; score not calibrated probability	Primary detector

One-Class SVM	Same features, scaled	Good for tight normal boundary on small data	$O(n^2)$ memory, slow on > 50k rows; sensitive to only y and v	Comparison
Autoencoder (Dense or LSTM)	24-h load windows per appliance	Learns complex temporal shape; reconstruction error pinpoints abnormal hours	Needs more data and tuning; threshold selection; less explainable	Advanced variant for appliance-shape anomalies

Anomaly logic: combine (a) Isolation Forest score, (b) forecast residual z-score > 3, (c) rule checks (appliance on > 12 h, night base load > $2 \times$ median). Final severity = max of normalized scores. Evaluation uses simulator fault injection as labeled ground truth (precision, recall, F1).

8.3 Consumption classification

Model	Advantages	Disadvantages	Role
Decision Tree	Fully interpretable rules; shows as a tree in UI/report	Overfits; lower accuracy	Explainability baseline
Random Forest (recommended)	Robust, little tuning, feature importance, handles imbalance with class weights	Larger model; less interpretable than single tree	Primary classifier
XGBoost	Often best accuracy; SHAP support	More hyperparameters; can overfit small data	Comparison / final if +3% F1

- Task A: household-day $\rightarrow$ Low / Normal / High (labels per Section 5.3 step 9).
- Task B: household $\rightarrow$ Efficient / Average / Heavy tier using monthly features (kWh per person, peak share, base load, weather sensitivity).
- Features: daily kWh, peak share, base load, load factor, temperature, weekday, occupancy, top-appliance share.
- Handle class imbalance with `class_weight='balanced'` or SMOTE on train only; report macro-F1.

8.4 Recommendation system

Approach	How it works here	Advantages	Disadvantages
----------	-------------------	------------	---------------

Rule-based	~30 expert rules: IF condition on analytics/ML output THEN recommendation template with savings formula	Transparent, works day 1, no training data needed	Static; doesn't personalize by preference
Content-based filtering	Recommendations and households described by feature vectors (appliance types, peak pattern, segment, effort tolerance); cosine similarity + past accepted recs	Personalizes without other users; explainable ("similar to tips you accepted")	Limited novelty; needs item metadata
Collaborative filtering	Matrix factorization on household × recommendation acceptance	Discovers non-obvious tips	Needs many users and interactions (cold start); not feasible with one dataset
Hybrid (recommended)	Rules + ML triggers generate candidates → content-based similarity and effectiveness history re-rank → top 5	Best of both; explainable; improves with feedback	More components to build

Candidate generation → scoring → ranking pipeline:

1. Candidates from rules engine, anomaly triggers, forecast-over-budget triggers, classification = High.
2. Savings Engine computes kWh/cost/CO2 for each candidate.
3. Score per FR-REC-6; content-based similarity boosts categories the user accepted before; ineffective categories (TRK) are down-weighted.
4. Filter duplicates and recently rejected; return top 5 with explanation.

Sample rules:

Rule ID	Condition	Recommendation	Savings formula
R-AC-01	AC share > 35% and avg daily AC hours > 6	Raise AC setpoint from 22°C to 24°C	AC kWh × 6% per °C × 2
R-AC-02	AC running 01:00–05:00 on > 50% nights	Use sleep timer to turn off at 02:00	AC kW × 3 h × nights

R-SHIFT-01	Washing machine/dishwasher used 18:00–22:00 and TOU tariff	Shift to after 22:00	kWh unchanged; cost × (peak – off-peak rate)
R-STBY-01	Night base load > 150 W	Unplug/smart-plug standby devices	(base – 80 W) × 8 h × 30
R-FRIDGE-01	Fridge efficiency index < 0.7 or anomaly on compressor duty cycle	Check door seal / consider 5-star model	Current – benchmark kWh
R-GEY-01	Geyser on > 2 h/day	Set timer to 30 min before use	Geyser kW × (hours – 0.75) × 30
R-LIGHT-01	Lighting kWh > benchmark × 1.3	Switch to LED	Lighting kWh × 0.5

8.5 Explainability

- SHAP TreeExplainer for XGBoost/Random Forest; top 3 drivers converted to text (e.g., "High temperature (+3.1 kWh), weekend (+1.2 kWh)").
- Prophet component plots shown in Forecast Insights.
- Anomalies show expected vs. actual chart and contributing features.
- Every recommendation cites its trigger (rule ID or model output) and evidence.

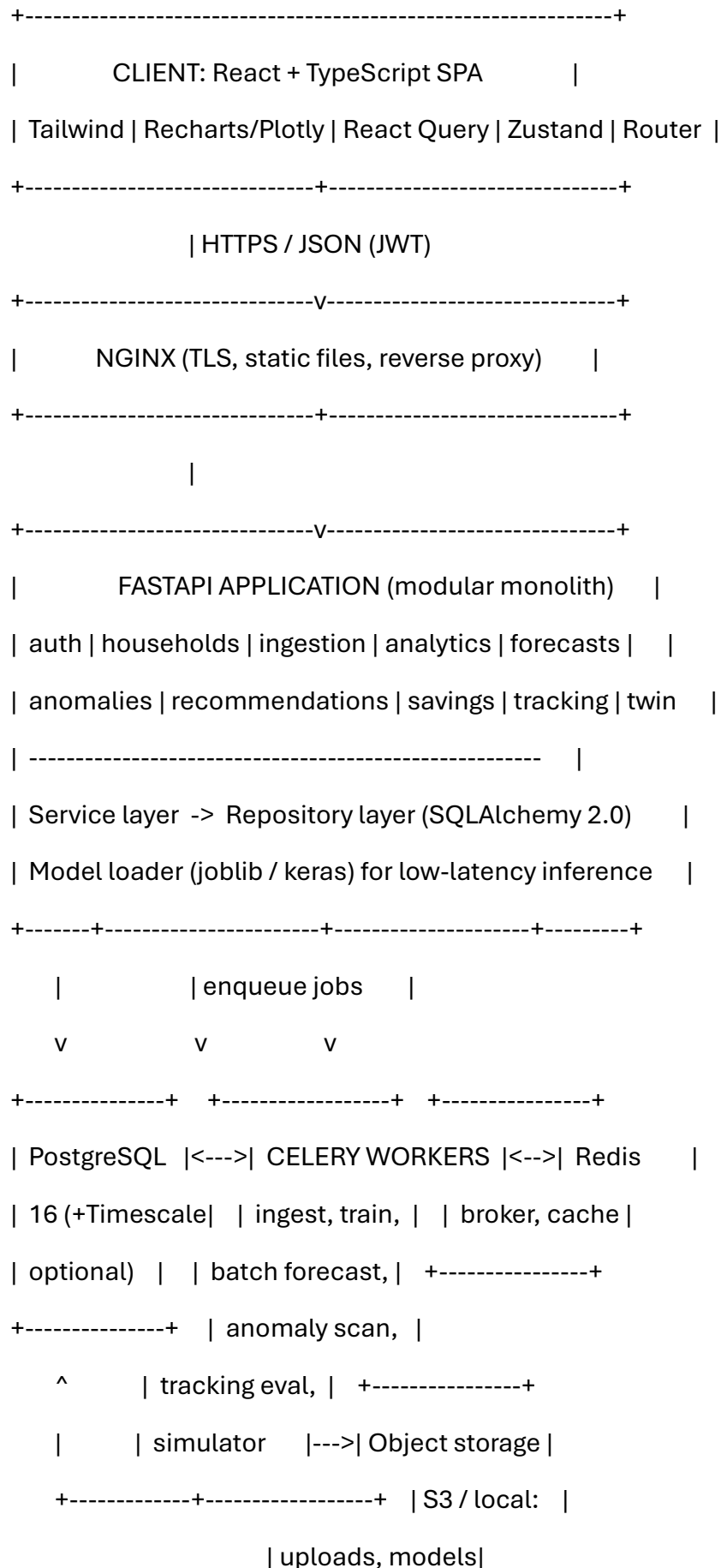
8.6 MLOps (project scale)

- Experiment tracking: MLflow (local file store); artifacts saved with joblib / .keras.
- Registry table (ml_models) marks the active version; API loads it at startup and on promotion.
- Retraining: weekly Celery beat job; champion/challenger comparison on last 30 days.
- Drift: weekly MAPE on actuals and PSI on key features; alert admin above thresholds.

9. Software Architecture & Tech Stack

Decision: modular monolith — one FastAPI codebase with strict module boundaries, plus a separate Celery worker container for ML and ingestion jobs. It gives microservice-like separation where it matters (heavy ML off the request path) without the operational cost a student team can't absorb.

9.1 High-level architecture



+-----+

MLflow (experiments) Prometheus + Grafana (metrics)

9.2 Monolith vs. microservices

Criterion	Monolith (modular)	Microservices
Team size fit (3–5 students)	Excellent	Poor — overhead per service
Development speed	Fast; one repo, one deploy	Slower; contracts, service discovery
Deployment complexity	1 API + 1 worker + DB + Redis	8–10 services, gateway, orchestration (K8s)
Independent scaling	Partial (worker scales separately)	Full
Fault isolation	Moderate	High
Debugging	Simple stack traces	Distributed tracing needed
Cost on AWS	~1 t3.medium + db.t3.micro	Several instances or EKS
Future migration	Modules can be extracted along existing boundaries	N/A

9.3 Tech stack with justification

Layer	Choice	Why
Frontend framework	React 18 + TypeScript + Vite	Component model, type safety, fast builds, large ecosystem
Styling	Tailwind CSS + shadcn/ui	Rapid consistent UI, accessible primitives
Charts	Recharts (standard charts), Plotly.js (heatmaps, forecast bands, zoom)	Recharts is React-native and light; Plotly for scientific charts
State / data fetching	TanStack React Query + Zustand	Server-state caching; minimal global state

Forms	React Hook Form + Zod	Validation mirrors backend Pydantic schemas
Backend	FastAPI (Python 3.11)	Same language as ML; async; Pydantic validation; auto OpenAPI; high performance
Backend alternative	Django + DRF	Built-in admin/ORM, but heavier and sync-first; choose only if team knows it well
ORM / migrations	SQLAlchemy 2.0 + Alembic	Mature, typed, migration history
Database	PostgreSQL 16 (+ TimescaleDB optional)	Relational integrity, JSONB for scenario params, partitioning, window functions
Async jobs	Celery + Redis	Offload training, bulk ingest, batch forecasts
ML	scikit-learn, XGBoost, Prophet, statsmodels, TensorFlow/Keras (LSTM, autoencoder), SHAP, Optuna	Standard, well- documented, laptop- friendly
Data processing	Pandas, NumPy	Resampling, feature engineering
Notebooks / EDA	Jupyter, Plotly, Seaborn	Exploration and report figures
Testing	pytest, httpx, Vitest, React Testing Library, Playwright (E2E)	Unit + API + UI coverage
Containerization	Docker + Docker Compose	Reproducible dev and deploy
CI/CD	GitHub Actions	Lint, test, build, push image
Cloud (recommended)	AWS: EC2 (Compose), RDS PostgreSQL, S3, CloudWatch; Azure	Free-tier/credits friendly

equivalent: App Service, Azure
Database for PostgreSQL, Blob

9.4 Backend project structure

```
backend/  
  
  app/  
  
    main.py  
  
    core/      # config, security, db session, logging  
  
    modules/  
  
      auth/    # router.py, schemas.py, service.py, repository.py  
  
      households/  
  
      ingestion/  
  
      analytics/  
  
      forecasting/  
  
      anomalies/  
  
      classification/  
  
      recommendations/  
  
      savings/  
  
      tracking/  
  
      digital_twin/  
  
      simulator/  
  
  ml/  
  
    features.py # shared feature engineering  
  
    training/  # train_forecast.py, train_anomaly.py, ...  
  
    registry.py  
  
    workers/   # celery_app.py, tasks.py  
  
  alembic/  
  
  tests/  
  
frontend/
```

src/{pages,components,features,api,hooks,lib}

ml-notebooks/

docker-compose.yml

10. Database Design

The schema has the 8 required tables plus 5 supporting tables (households, tariffs, ingestion jobs, anomalies, ML models). A user owns households; everything else hangs off household_id, which is also the privacy boundary.

erDiagram

USERS ||--o{ HOUSEHOLDS : owns

HOUSEHOLDS ||--o{ APPLIANCES : has

HOUSEHOLDS ||--o{ CONSUMPTION_DATA : records

APPLIANCES ||--o{ CONSUMPTION_DATA : measures

HOUSEHOLDS ||--o{ FORECASTS : predicts

HOUSEHOLDS ||--o{ ANOMALIES : flags

HOUSEHOLDS ||--o{ RECOMMENDATIONS : receives

RECOMMENDATIONS ||--o{ RECOMMENDATION_TRACKING : tracked_by

HOUSEHOLDS ||--o{ SAVINGS_REPORTS : summarizes

HOUSEHOLDS ||--o{ DIGITAL_TWIN_SCENARIOS : simulates

TARIFFS ||--o{ HOUSEHOLDS : billed_by

INGESTION_JOBS ||--o{ CONSUMPTION_DATA : loads

10.1 Tables

users

Column	Type	Constraints
user_id	UUID	PK
email	VARCHAR(255)	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	NOT NULL
full_name	VARCHAR(120)	

role	ENUM(user, auditor, admin)	DEFAULT 'user'
consent_research	BOOLEAN	DEFAULT false
created_at, updated_at	TIMESTAMPTZ	NOT NULL
deleted_at	TIMESTAMPTZ	NULL (soft delete)

households (supporting)

Column	Type	Constraints
household_id	UUID	PK
user_id	UUID	FK → users, NOT NULL
name	VARCHAR(100)	
dwelling_type	ENUM(house, apartment, hostel)	
occupants	SMALLINT	CHECK > 0
area_sqft	INT	
city, country	VARCHAR(80)	
tariff_id	UUID	FK → tariffs
emission_factor_kg_per_kwh	NUMERIC(5,3)	DEFAULT 0.710
monthly_budget_kwh	NUMERIC(8,2)	NULL
created_at	TIMESTAMPTZ	

tariffs (supporting)

Column	Type	Constraints
tariff_id	UUID	PK
name	VARCHAR(100)	
type	ENUM(flat, slab, tou)	
currency	CHAR(3)	e.g. INR, USD
rate_config	JSONB	slabs [{upto_kwh, rate}] or TOU [{start, end, rate}]
fixed_charge	NUMERIC(8,2)	

appliances

Column	Type	Constraints
appliance_id	UUID	PK
household_id	UUID	FK → households, NOT NULL
name	VARCHAR(80)	e.g. "Bedroom AC"
category	ENUM(ac, fridge, washing_machine, dishwasher, geyser, lighting, tv, microwave, furnace, ev_charger, other)	
rated_power_w	INT	CHECK > 0
standby_power_w	INT	DEFAULT 0
star_rating	SMALLINT	1–5, NULL
purchase_year	SMALLINT	
source_column	VARCHAR(80)	mapping to dataset column
is_active	BOOLEAN	DEFAULT true

consumption_data (partitioned by month on ts)

Column	Type	Constraints
consumption_id	BIGSERIAL	PK (with ts)
household_id	UUID	FK → households, NOT NULL
appliance_id	UUID	FK → appliances, NULL = whole-house total
ts	TIMESTAMPTZ	NOT NULL (15-min bucket start)
energy_kwh	NUMERIC(10,4)	CHECK ≥ 0
avg_power_kw	NUMERIC(8,4)	

temperature_c, humidity_pct	NUMERIC(5,2)	
weather_code	VARCHAR(30)	
occupancy	SMALLINT	
source	ENUM(csv, manual, simulator)	
ingestion_job_id	UUID	FK → ingestion_jobs
is_imputed, outlier_flag	BOOLEAN	DEFAULT false
		UNIQUE (household_id, appliance_id, ts); INDEX (household_id, ts)

ingestion_jobs (supporting)

Column	Type	Constraints
job_id	UUID	PK
household_id	UUID	FK → households
source	ENUM(csv, manual, simulator)	
file_name, file_sha256	VARCHAR	
status	ENUM(queued, running, succeeded, failed)	
rows_total, rows_valid, rows_rejected	INT	
quality_report	JSONB	
created_at, finished_at	TIMESTAMPTZ	

forecasts

Column	Type	Constraints
forecast_id	UUID	PK

household_id	UUID	FK → households
appliance_id	UUID	FK → appliances, NULL = total
model_id	UUID	FK → ml_models
run_at	TIMESTAMPTZ	
granularity	ENUM(hour, day)	
target_ts	TIMESTAMPTZ	
predicted_kwh	NUMERIC(10,4)	
lower_80, upper_80, lower_95, upper_95	NUMERIC(10,4)	
actual_kwh	NUMERIC(10,4)	NULL until known
explanation	JSONB	SHAP top features
		INDEX (household_id, target_ts)

anomalies (supporting)

Column	Type	Constraints
anomaly_id	UUID	PK
household_id	UUID	FK → households
appliance_id	UUID	FK → appliances, NULL
start_ts, end_ts	TIMESTAMPTZ	
expected_kwh, actual_kwh	NUMERIC(10,4)	
score	NUMERIC(4,3)	0–1
severity	ENUM(low, medium, high)	
detector	VARCHAR(40)	isolation_forest / residual / rule

user_feedback	ENUM(expected, real_issue)	NULL
---------------	----------------------------	------

status	ENUM(open, acknowledged, resolved)
--------	------------------------------------

recommendations

Column	Type	Constraints
recommendation_id	UUID	PK
household_id	UUID	FK → households
appliance_id	UUID	FK → appliances, NULL
anomaly_id	UUID	FK → anomalies, NULL
rule_code	VARCHAR(30)	e.g. R-AC-01
category	ENUM(appliance, behavior, schedule, upgrade, anomaly)	
source	ENUM(rule, ml, twin)	
title, description	VARCHAR / TEXT	
evidence	JSONB	data behind the rec
est_kwh_month, est_cost_month, est_co2_kg_month	NUMERIC(10,2)	
effort	ENUM(low, medium, high)	
confidence, rank_score	NUMERIC(4,3)	
status	ENUM(generated, viewed, accepted, rejected, snoozed, completed, expired)	

created_at, expires_at TIMESTAMPTZ

recommendation_tracking

Column	Type	Constraints
tracking_id	UUID	PK
recommendation_id	UUID	FK → recommendations, UNIQUE
action	ENUM(accepted, rejected, snoozed)	
action_at	TIMESTAMPTZ	
reason	TEXT	
baseline_start, baseline_end	DATE	
baseline_kwh	NUMERIC(10,2)	weather-normalized
eval_start, eval_end	DATE	
counterfactual_kwh, actual_kwh	NUMERIC(10,2)	
realized_kwh_saved, realized_cost_saved	NUMERIC(10,2)	
improvement_score	NUMERIC(6,2)	%
effectiveness_score	NUMERIC(6,2)	realized / projected %
effectiveness_label	ENUM(effective, partial, ineffective, pending)	

savings_reports

Column	Type	Constraints
report_id	UUID	PK
household_id	UUID	FK → households
period_start, period_end	DATE	UNIQUE (household_id, period_start)

baseline_kwh, actual_kwh	NUMERIC(10,2)
projected_saved_kwh, realized_saved_kwh	NUMERIC(10,2)
cost_saved	NUMERIC(10,2)
co2_saved_kg	NUMERIC(10,2)
recs_accepted, recs_effective	INT
household_improvement_score	NUMERIC(5,2) 0–100
generated_at	TIMESTAMPTZ

digital_twin_scenarios

Column	Type	Constraints
scenario_id	UUID	PK
household_id	UUID	FK → households
name	VARCHAR(100)	
baseline_snapshot	JSONB	calibrated appliance params
changes	JSONB	e.g. [{appliance_id, field: "hours_per_day", from: 8, to: 6}]
horizon_days	SMALLINT	DEFAULT 30
baseline_kwh, scenario_kwh	NUMERIC(10,2)	
kwh_saved, cost_saved, co2_saved_kg	NUMERIC(10,2)	
peak_kw_before, peak_kw_after	NUMERIC(8,3)	
result_series	JSONB	daily projected curve
converted_recommendation_id	UUID	FK → recommendations, NULL
created_at	TIMESTAMPTZ	

ml_models (supporting)

Column	Type	Constraints
--------	------	-------------

model_id	UUID	PK
task	ENUM(forecast, anomaly, classification)	
algorithm	VARCHAR(40)	
version	VARCHAR(20)	
scope	VARCHAR(20)	global / household
household_id	UUID	FK → households, NULL
metrics	JSONB	MAE, RMSE, MAPE, F1...
artifact_uri	TEXT	
is_active	BOOLEAN	
trained_at	TIMESTAMPTZ	

10.2 Views and indexes

- Materialized views: mv_hourly_consumption, mv_daily_consumption, mv_monthly_appliance_summary, refreshed after ingestion.
- Composite indexes on (household_id, ts) for every time-series table; BRIN index on consumption_data.ts.
- All FKs ON DELETE CASCADE from households to support full data deletion.

11. API Design

REST over JSON, versioned at /api/v1, JWT Bearer auth on all routes except /auth/*. Household-scoped routes verify the caller owns (or is an assigned auditor of) {hid}. Errors use RFC 7807 problem+json; lists use ?page=&size= pagination.

Met hod	Endpoint	Purpose	Module
POST	/auth/register	Create account	Auth
POST	/auth/login	Returns access + refresh token	Auth
POST	/auth/refresh	Rotate tokens	Auth

GET / PATC H	/users/me	Profile, consent	Auth
GET / POS T	/households	List / create households	Storage
GET / PATC H / DEL ETE	/households/{hid}	Household detail; DELETE purges data	Storage
GET / POS T	/households/{hid}/appliances	Appliance inventory	Storage
PATC H / DEL ETE	/households/{hid}/appliances /{aid}	Edit / deactivate appliance	Storage
POS T	/households/{hid}/ingest/csv	Multipart upload + mapping template id → job_id	Ingestion
GET	/ingest/jobs/{job_id}	Status, quality report, error file URL	Ingestion
POS T	/households/{hid}/ingest/manual	Manual entries (array)	Ingestion
POS T	/households/{hid}/simulator/runs	Start simulation {days, resolution, profile, faults[], stream}	Ingestion
DEL ETE	/simulator/runs/{run_id}	Stop streaming run	Ingestion

GET	/households/{hid}/consumption	Time series ?from&to&granularity=hour day week month&appliance_id	Analytics
GET	/households/{hid}/analytics/summary	KPIs: total, avg, change %, cost, peak, base load	Analytics
GET	/households/{hid}/analytics/peaks	Peak windows + hour×weekday heatmap	Analytics
GET	/households/{hid}/analytics/appliances	Appliance breakdown and efficiency index	Analytics
GET	/households/{hid}/analytics/segments	Load profile segment + distribution	Analytics
GET	/households/{hid}/analytics/cost	Tariff-based cost breakdown, TOU split	Analytics
GET	/households/{hid}/forecasts	?horizon=48h 7d 30d&appliance_id → series with intervals + explanation	ML
POST	/households/{hid}/forecasts/run	Trigger fresh forecast (async)	ML
GET	/households/{hid}/anomalies	?status&severity&from&to	ML
PATCH	/anomalies/{id}	Acknowledge / feedback	ML
GET	/households/{hid}/classification	Daily labels + household tier + SHAP drivers	ML
GET	/households/{hid}/recommendations	Active ranked recommendations	Recommendation
POST	/households/{hid}/recommendations/generate	Re-run engine	Recommendation
POST	/recommendations/{rid}/actions	{action: accept reject snooze, reason}	Tracking
GET	/households/{hid}/tracking	Tracking records + effectiveness	Tracking

GET	/households/{hid}/savings	?period=month realized vs projected, cost, CO2	Savings
POST	/savings/estimate	Stateless estimate for an arbitrary change	Savings
GET	/households/{hid}/twin	Calibrated twin model (appliance params, calibration error)	Digital Twin
POST	/households/{hid}/twin/simulate	Run what-if {changes[], horizon_days} → results (not saved)	Digital Twin
GET	/households/{hid}/twin/scenarios	List / save scenarios	Digital Twin
POST	/twin/scenarios/{sid}/convert	Scenario → tracked recommendation	Digital Twin
GET	/households/{hid}/reports/monthly.pdf	PDF report export	Dashboard
POST	/admin/models/train	{task, algorithm, scope} → job	Admin
GET	/admin/models	Registry with metrics	Admin
POST	/admin/models/{id}/activate	Promote model	Admin
GET	/health, /metrics	Liveness, Prometheus	Ops

11.1 Sample: what-if simulation

POST /api/v1/households/{hid}/twin/simulate

```
{
  "horizon_days": 30,
  "changes": [
    {"appliance_id": "ac-bedroom", "field": "hours_per_day", "to": 6},
    {"appliance_id": "washer", "field": "cycles_per_day", "to": 1}
  ]
}
```

```
}
```

200 OK

```
{
  "baseline": {"kwh": 412.5, "cost": 2887.50, "co2_kg": 292.9, "peak_kw": 3.4},
  "scenario": {"kwh": 346.8, "cost": 2427.60, "co2_kg": 246.2, "peak_kw": 3.4},
  "delta": {"kwh": -65.7, "cost": -459.90, "co2_kg": -46.7, "pct": -15.9},
  "by_appliance": [{"appliance_id": "ac-bedroom", "kwh_delta": -54.0}, ...],
  "series": [{"date": "2026-10-01", "baseline_kwh": 13.8, "scenario_kwh": 11.6}, ...],
  "assumptions": ["Tariff: slab, avg 7.00 INR/kWh", "Emission factor 0.71 kg/kWh"]
}
```

11.2 Sample: recommendation object

```
{
  "recommendation_id": "b3f...",
  "title": "Raise bedroom AC setpoint to 24°C",
  "category": "appliance",
  "source": "rule",
  "rule_code": "R-AC-01",
  "reason": "AC used 8.2 h/day and is 41% of your consumption this month",
  "evidence": {"ac_share": 0.41, "avg_hours": 8.2, "avg_setpoint_c": 22},
  "estimate": {"kwh_month": 21.6, "cost_month": 151.2, "co2_kg_month": 15.3},
  "range_pct": [8, 14],
  "effort": "low",
  "confidence": 0.82,
  "rank_score": 0.74,
  "status": "generated"
}
```

12. Dashboard Design

Ten dashboards sit in a left navigation; a global filter bar (household, date range, granularity, appliance multi-select) persists across them via URL query params. Every chart has a "Why?" tooltip linking to its explanation.

#	Dashboard	KPIs	Charts	Filters	User actions
1	Executive Overview	This month kWh, cost, % vs last month, forecast month-end kWh and bill, CO2, open anomalies, household improvement score (0–100), efficiency tier	KPI cards with sparklines; 30-day line (actual + forecast band); appliance share donut; top 3 recommendations card	Household, month	Drill into any KPI; accept top recommendation; view anomaly
2	Daily Consumption	Today kWh, peak kW and time, base load W, vs 7-day avg	24-h line at 15-min grain (actual vs expected); stacked area by appliance; anomaly markers	Date picker, appliance, resolution (15/60 min)	Click anomaly marker → detail and feedback; compare with another day
3	Weekly Consumption	Week kWh, avg/day, weekday vs weekend ratio, peak day	7-day bars (stacked by appliance); hour × weekday heatmap; week-over-week line	Week, appliance	Toggle appliances; highlight peak windows; export CSV

4	Monthly Consumption	Month kWh, cost, kWh/occupant, vs same month last year, weather-normalized change	Daily bars with temperature overlay line; 12-month trend line; kWh vs temperature scatter with regression	Month/year, normalize by weather toggle	Download PDF report; set monthly budget
5	Appliance-Level Performance	Top consumer, # appliances above benchmark, standby kWh, efficiency index per appliance	Ranked horizontal bars (kWh, cost); appliance vs benchmark bullet chart; usage timeline (Gantt of on-periods); run hours/cycles table	Appliance category, date range, metric (kWh/cost/hours)	Edit appliance details; open appliance in Digital Twin; see appliance recommendations
6	Forecast Insights	Next 7-day kWh, month-end projection vs budget, model used, validation MAPE	Actual + forecast line with 80/95% bands; Prophet components (trend, weekly, yearly); SHAP driver bars; forecast-vs-actual history	Horizon (48h/7d/30d), appliance, model (auto/XGBoost/Prophet/LSTM)	Re-run forecast; compare models; set budget alert
7	Cost Analytics	Bill to date, projected bill, avg cost/kWh, peak vs off-peak cost	Cost by appliance bars; TOU split stacked bar; slab progression	Tariff, period, appliance	Change tariff plan; simulate tariff switch; export

		share, most expensive appliance	gauge; monthly bill trend		
8	Savings Analytics	Realized kWh saved, cost saved, CO2 avoided, projected vs realized %, equivalents (trees)	Cumulative savings line (realized vs projected); savings by recommendation category bars; monthly savings waterfall	Period, category	Download savings report; share badge
9	Recommendation Center	Active recs, acceptance rate, avg effectiveness, total potential savings	Recommendation cards (title, reason, savings, effort, confidence); effectiveness table with before/after mini-charts; acceptance funnel (generated → viewed → accepted → effective)	Status, category, appliance, effort	Accept / reject / snooze with reason; "Simulate this"; mark completed; view evidence
10	Digital Twin Simulator	Calibration error %, baseline vs scenario kWh, cost, CO2, peak kW, % change	Appliance lever panel (sliders, toggles, dropdowns); baseline vs scenario 30-day line; delta by appliance bars; scenario	Horizon (7/30/365 days), weather scenario (normal/hot/cold)	Adjust levers; save scenario; compare scenarios; convert to recommendation; reset

comparison
table (up to 3)

12.1 Layout principles

- Top row: 4–6 KPI cards; middle: primary chart full width; bottom: 2-column secondary charts and tables.
- Color: one hue per appliance category, consistent across all dashboards; red reserved for anomalies/overruns, green for savings.
- Empty states guide action ("Upload data or start the simulator").
- Admin-only extra page: **Model Performance** (metrics per model, drift chart, retrain button).

12.2 Wireframe: Executive Overview

```
+-----+
| [Household v] [Sep 2026 v]          (bell) (avatar) |
+-----+-----+-----+-----+-----+-----+
| 312 kWh | INR 2,184 | -8% vs Aug| Forecast | 221 kg CO2| Score |
| ~spark~ | ~spark~ |   | 405 kWh |   | 72/100 |
+-----+-----+-----+-----+-----+-----+
| Daily kWh: actual ----- forecast ..... with band           |
+-----+-----+-----+-----+-----+-----+
| Appliance share (donut)    | Top recommendations           | | |
| AC 41% | Fridge 14% | Geyser .. | 1. Raise AC to 24C -22 kWh |
|                               | 2. Geyser timer   -15 kWh  |
|                               | [Accept] [Simulate]      |
+-----+-----+-----+-----+-----+-----+
| Open anomalies (2): Fridge running 23 h on Sep 21 [View]      |
+-----+-----+-----+-----+-----+-----+
```

13. Digital Twin Design

The digital twin is a calibrated, physics-informed simulation of the household: each appliance is a parameterized model whose parameters are learned from the household's history, so changing a parameter predicts the new consumption. It is a

hybrid of bottom-up appliance models and the ML forecaster (for weather and residual load).

13.1 Architecture

flowchart TD

A[Historical consumption
+ appliance inventory] --> B[Parameter learner]

B --> C[Virtual household
appliance models]

W[Weather forecast /
scenario] --> C

C --> D[Calibration check
within 10% of actual]

D --> E[Baseline projection]

U[User what-if levers] --> F[Scenario engine]

E --> F

F --> G[Savings engine
kWh, cost, CO2, peak]

G --> H[Results + save
convert to rec]

History calibrates the virtual appliances; user levers modify a copy of the baseline and the savings engine prices the difference.

13.2 Appliance model

Each virtual appliance i has: rated power P_i (kW), duty cycle d_i (fraction of on-time the compressor/element draws full power), hours per day h_i , cycles per day c_i and energy per cycle e_i (for cycle-based devices), standby power s_i , schedule (hour-of-day probability vector), temperature sensitivity k_i (for AC/heater), efficiency factor η_i .

$$E_{\{i, \text{day}\}} = \frac{P_i \cdot d_i \cdot h_i}{(1 + k_i \cdot \max(0, T - T_{\text{ref}})) \eta_i} + c_i e_i + s_i (24 - h_i)$$

$$E_{\{\text{house, day}\}} = \sum_i E_{\{i, \text{day}\}} + E_{\{\text{residual}\}}$$

E_{residual} (unmetered/unknown loads) comes from the ML forecaster on total load minus the sum of modeled appliances.

13.3 Calibration (learning parameters)

1. **Hours/day and schedule:** from appliance series, on-state = kW > 10% of P_i ; average on-hours and hourly on-probability over last 30 days.
2. **Duty cycle:** mean kW while on / P_i .
3. **Cycles and energy per cycle:** on-segment detection for washer/dishwasher; kWh per segment.

4. **Temperature sensitivity k_i** : linear regression of AC daily kWh on cooling degree days.
5. **Standby**: median kW while "off".
6. **Validate**: simulate last 30 days; if $|\text{simulated} - \text{actual}| / \text{actual} > 10\%$, scale residual and flag "low confidence". Manual-entry households without appliance series use rated power $\times$ user-entered hours with default duty cycles by category.

13.4 Scenario levers

Lever	Parameter changed	Example
Usage hours	h_i	AC 8 $\rightarrow$ 6 h/day
Cycles	c_i	Washer 2 $\rightarrow$ 1 cycle/day
Setpoint	d_i and k_i via $\sim 6\%$ kWh per $^{\circ}\text{C}$ rule	AC 22 $\rightarrow$ 24 $^{\circ}\text{C}$
Schedule shift	schedule vector (affects cost under TOU and peak kW, not kWh)	Dishwasher 20:00 $\rightarrow$ 23:00
Upgrade	η_i , P_i from efficiency catalog	3-star $\rightarrow$ 5-star fridge
Standby elimination	$s_i \rightarrow 0$	Smart plugs on TV/set-top box
Add/remove appliance	new appliance model	Add EV charger 7 kW $\times$ 2 h
Solar	subtract PV generation profile	2 kWp rooftop
Weather	T series	Hot summer (+3 $^{\circ}\text{C}$)

13.5 Worked example

Current state (calibrated): AC rated 1.5 kW, duty cycle 0.7, 8 h/day $\rightarrow$ 8.4 kWh/day. Washing machine 0.5 kWh/cycle, 2 cycles/day $\rightarrow$ 1.0 kWh/day. Tariff ₹7.00/kWh flat; emission factor 0.71 kg CO₂/kWh.

Scenario: reduce AC by 2 h/day (8 $\rightarrow$ 6 h).

Metric (30 days)	Current	Scenario	Change
AC energy	252.0 kWh	189.0 kWh	-63.0 kWh (-25%)
Washing machine energy	30.0 kWh	30.0 kWh	0

AC cost	₹1,764	₹1,323	-₹441
AC CO2	178.9 kg	134.2 kg	-44.7 kg
Annualized (AC months only, ~6/yr)	—	—	-378 kWh, -₹2,646, -268 kg CO2

Calculation: $1.5 \text{ kW} \times 0.7 \times 2 \text{ h} \times 30 \text{ days} = 63 \text{ kWh}$; $63 \times ₹7 = ₹441$; $63 \times 0.71 = 44.7 \text{ kg}$. Under slab tariffs the saving is priced at the marginal (highest) slab reached, which makes it larger.

13.6 Implementation strategy

1. **Phase 5a (week 12):** ApplianceModel and HouseholdTwin Python classes (dataclasses), vectorized with NumPy to simulate 365 days $\times$ 24 h in $< 200 \text{ ms}$.
2. **Phase 5b:** calibration service reading mv_hourly_consumption; store calibrated snapshot as JSONB.
3. **Phase 5c (week 13):** /twin/simulate endpoint (synchronous; result cached in Redis by hash of changes for 10 min); scenario save/compare/convert.
4. **Phase 5d:** React simulator UI with debounced sliders (300 ms) for near-real-time results.
5. **Validation:** back-test by taking a real historical change in the dataset (e.g., a month where AC hours dropped) and checking twin-predicted vs. actual delta; target error $\leq 15\%$.
6. **Reuse:** the same simulator engine powers the synthetic data generator (FR-ING-8) with added Gaussian noise and fault injection, so one codebase serves both.

14. User Flows, Data Flow & API Flow

The core loop is: bring data in $\rightarrow$ understand it $\rightarrow$ act on a recommendation $\rightarrow$ see whether it worked.

14.1 User journey (primary persona)

flowchart LR

A[Sign up +
consent] --> B[Create household
+ appliances]

B --> C{Data source}

C -->|CSV| D[Upload + map
columns]

C -->|Manual| E[Enter usage]

C -->|Demo| F[Run simulator]

D --> G[Quality report]

E --> G

F --> G

G --> H[Executive
Overview]

H --> I[Review recs
+ anomalies]

I --> J[Simulate in
digital twin]

J --> K[Accept rec]

K --> L[Track 14 days
+ savings report]

A new user reaches the Executive Overview in under 5 minutes; the loop from recommendation to measured effectiveness takes 14 days.

14.2 Key flows (step-by-step)

Flow A — CSV onboarding

1. User uploads Kaggle CSV → system detects columns and proposes Kaggle mapping template.
2. User confirms appliance mapping (e.g., Fridge [kW] → Fridge, category fridge, rated 150 W).
3. Job queued; progress bar; quality report shows 503k rows, 0.2% imputed, 14 outliers flagged (illustrative).
4. On completion, worker refreshes views, trains/selects forecast model, runs anomaly scan, classification and recommendation generation.
5. Notification: "Your insights are ready" → Executive Overview.

Flow B — Anomaly to action

1. Streaming simulator injects "fridge compressor stuck on".
2. Anomaly detector flags high severity within 5 s; bell notification.
3. User opens anomaly: expected 1.2 kWh vs actual 3.6 kWh; marks "real issue".
4. Recommendation R-FRIDGE-01 generated with estimated savings; user accepts → tracking starts.

Flow C — Recommendation lifecycle

stateDiagram-v2

[*] --> Generated

Generated --> Viewed

Viewed --> Accepted

Viewed --> Rejected

Viewed --> Snoozed

Snoozed --> Viewed: after 7 days

Accepted --> Evaluating: baseline captured

Evaluating --> Effective: $\geq 70\%$

Evaluating --> Partial: 30-70%

Evaluating --> Ineffective: $< 30\%$

Generated --> Expired: 30 days

Rejected --> [*]

Every accepted recommendation ends with an effectiveness label that feeds back into ranking.

14.3 Data flow

flowchart LR

S1[CSV] --> V[Validate +
map]

S2[Manual form] --> V

S3[Simulator] --> V

V --> P[Clean, impute,
flag outliers]

P --> DB[(consumption_data)]

DB --> MV[Hourly / daily
views]

MV --> FE[Feature
engineering]

FE --> ML[Forecast, anomaly,
classify]

ML --> R[Recommend +
estimate savings]

R --> DB2[(recs, forecasts,
anomalies)]

DB2 --> API[REST API]

MV --> API

API --> UI[Dashboards]

Raw data is validated and cleaned once at ingestion; every downstream consumer reads the same cleaned tables and views.

14.4 API flow: accept a recommendation

sequenceDiagram

participant UI as React UI

participant API as FastAPI

participant DB as PostgreSQL

participant Q as Celery worker

UI->>API: POST /recommendations/{rid}/actions accept

API->>API: Verify JWT + household ownership

API->>DB: Update status, insert tracking row

API->>Q: enqueue capture_baseline(rid)

API-->>UI: 200 tracking record (pending)

Q->>DB: Compute 14-day weather-normalized baseline

Note over Q: Day +14 scheduled job

Q->>DB: Actual vs counterfactual, scores

Q->>DB: Update rank weights, savings report

UI->>API: GET /households/{hid}/tracking

API-->>UI: Effectiveness + before/after series

The request path stays fast; baseline capture and the 14-day evaluation run in the worker.

15. Project Timeline & Roadmap

Sixteen weeks, six phases, assuming a team of 4 (1 frontend, 1 backend, 2 ML/data) working ~15 h/week each. Phases overlap slightly so frontend work starts while models are trained.

Phase	Weeks	Scope	Deliverables	Exit criteria
1. Data Collection & Storage	1–3	Dataset download + EDA; preprocessing pipeline; DB schema + Alembic migrations;	EDA notebook + data quality report; preprocess.py; ER diagram; working	Kaggle data loaded at 15-min grain;

		auth; CSV upload, manual entry, basic simulator; Docker Compose skeleton	ingestion API; seeded DB with Kaggle data	upload of 500k rows < 60 s
2. Analytics Dashboard	4–6	Analytics engine (trends, peaks, appliance analysis, segmentation, cost); React app shell, auth screens, global filters; dashboards 1–5 and 7	Analytics API; 6 live dashboards; UI design system	Dashboards render 1 year of data with p95 API < 500 ms
3. Machine Learning Models	6–10	Forecasting (ARIMA, Prophet, XGBoost, LSTM) benchmark; anomaly (Isolation Forest, OCSVM, autoencoder); classification (DT, RF, XGBoost); SHAP; model registry; Celery jobs; Forecast Insights dashboard	Model comparison report with metrics tables; saved artifacts; ML APIs; dashboard 6; model cards	Forecast MAPE $\leq 15\%$ (daily); anomaly F1 ≥ 0.80 on injected faults; classifier macro-F1 ≥ 0.85
4. Recommendation Engine	10–12	Rules table (~30 rules); ML triggers; content-based re-ranking; savings engine; tracking engine; Recommendation Center + Savings dashboards	Recommendation + savings + tracking APIs; dashboards 8–9	Every rec has savings, reason, effort; tracking computes scores on simulated before/after
5. Digital Twin	12–14	Appliance models, calibration, scenario engine, comparison, convert-to-rec; simulator reuse;	Twin service + API; dashboard 10; calibration report	Calibration error $\leq 10\%$; simulate < 1 s; back-test error $\leq 15\%$

6. Evaluation & Deployment	14–16	Digital Twin dashboard		
		End-to-end tests; performance and security checks; user testing (5–10 users, SUS); AWS deployment; documentation; final report and demo	Deployed URL; test reports; user study results; project report; slide deck; demo video	All Must requirements pass; demo script runs end-to-end without errors

15.1 Milestones

Week Milestone

- 3 M1: Data in the database, ingestion working (review 1)
- 6 M2: Analytics dashboards live
- 10 M3: ML models benchmarked and integrated (review 2 / mid-term)
- 12 M4: Recommendations + savings + tracking live
- 14 M5: Digital twin live, feature freeze
- 16 M6: Deployed, final report and viva

15.2 Team split

Role	Owns
ML Engineer 1	Preprocessing, forecasting, digital twin calibration
ML Engineer 2	Anomaly detection, classification, recommendation logic, SHAP
Backend Engineer	FastAPI modules, DB, Celery, simulator, deployment
Frontend Engineer	React app, all dashboards, twin UI

16. Evaluation Metrics & Success Criteria

The project is successful when all Must requirements work end-to-end in the deployed app and the targets below are met on the held-out test period.

16.1 Model metrics

Area	Metric	Definition	Target
Forecasting	MAE	mean $ y - \hat{y} $ (kWh)	Beat seasonal-naive baseline by $\geq 20\%$
Forecasting	RMSE	$\sqrt{\text{mean } (y - \hat{y})^2}$ (kWh)	Beat seasonal-naive by $\geq 20\%$
Forecasting	MAPE	mean $ y - \hat{y} / y \times 100$ (use sMAPE if y near 0)	$\leq 15\%$ daily, $\leq 25\%$ hourly
Forecasting	Interval coverage	% of actuals inside 80% interval	75–85%
Anomaly	Precision / Recall / F1	Against simulator-injected faults	F1 ≥ 0.80 , precision ≥ 0.75
Classification	Accuracy	correct / total	$\geq 88\%$
Classification	Precision, Recall, F1 (macro)	per class, averaged	Macro-F1 ≥ 0.85
Classification	Confusion matrix	reported per model	High-class recall ≥ 0.85
Digital twin	Calibration error	$ \text{sim} - \text{actual} / \text{actual}$ (30 days)	$\leq 10\%$
Recommendation	Precision@5 (offline)	Recs matching expert-labeled opportunities	≥ 0.6

$$\text{MAPE} = \frac{100}{n} \sum_{t=1}^n \left| \frac{y_t - \hat{y}_t}{y_t} \right|$$

16.2 Recommendation & business metrics

Metric	Definition	Target
Acceptance rate	accepted / viewed	$\geq 40\%$
Effectiveness rate	effective / evaluated accepted	$\geq 60\%$
Savings achieved	realized kWh saved / projected kWh saved	$\geq 70\%$
Estimated energy reduction	projected monthly kWh reduction if top 5 recs adopted	10–20%

Estimated cost reduction	same, in currency	10–20% of bill
CO2 reduction	kWh saved × emission factor	Reported per household
User engagement	weekly active users, sessions/week, twin scenarios/user	≥ 3 sessions/week; ≥ 2 scenarios/user in study
Usability	System Usability Scale	≥ 70
Time to first insight	signup → Executive Overview with data	< 5 min

16.3 Success criteria (definition of done)

- All three ingestion paths load data; Kaggle dataset fully processed.
- Ten dashboards live with filters and interactive charts.
- Four forecasting models benchmarked; best deployed with MAPE ≤ 15%.
- Anomaly detection catches ≥ 80% of injected faults.
- Classifier macro-F1 ≥ 0.85 with SHAP explanations.
- Hybrid recommender outputs ranked, quantified, explained recommendations.
- Tracking engine produces before/after and effectiveness scores.
- Digital twin calibrated within 10% and reproduces the AC example.
- Deployed on cloud via Docker with HTTPS.
- Report, model cards, API docs and demo video complete.

17. Risks & Mitigation

The biggest risk is that a single public dataset cannot prove behavior change; the simulator and counterfactual tracking are the mitigation.

#	Risk	Likelihood	Impact	Mitigation
R1	Dataset covers one household / one year, limiting generalization and peer comparison	High	High	Use simulator to generate varied households; global model + per-household fine-tune; state limitation in report

R2	No real users to validate recommendation effectiveness	High	Medium	Simulated before/after with behavior-change models; small user study (5–10 people) with manual entry
R3	Dataset columns differ from assumptions (units, appliances, no AC)	Medium	Medium	Canonical schema + mapping templates; confirm in Week 1; map furnace/heating to HVAC category
R4	LSTM/autoencoder underperform or take too long to train	Medium	Low	Treated as benchmarks; XGBoost/Isolation Forest are primary; train on hourly not minute data
R5	Anomaly ground truth unavailable	High	Medium	Fault injection in simulator gives labels; user feedback loop
R6	Savings estimates overstated, losing user trust	Medium	High	Show ranges, cite assumptions, weather-normalize, compare to counterfactual forecast
R7	Scope creep across 9 modules	High	High	MoSCoW priorities; feature freeze Week 14; Could items dropped first
R8	Performance issues with minute-level data	Medium	Medium	Store at 15-min; materialized views; partitioning; cache summaries in Redis
R9	Cloud cost / credits run out	Low	Medium	Single EC2 + RDS free tier; Docker Compose; shut down between demos
R10	Security gaps (auth, file upload)	Medium	High	Pydantic validation, file-type/size limits, OWASP checklist, dependency scanning (pip-audit, npm audit)
R11	Team member unavailability	Medium	Medium	Module ownership with a named backup; shared docs; PR reviews

R12	Emission factor / tariff values incorrect for region	Medium	Low	Admin-configurable; defaults cited in report
-----	--	--------	-----	--

18. Future Enhancements

These are deliberately out of v1 scope and sequenced by value versus effort.

Enhancement	Description	Value
Live IoT integration	MQTT ingestion from smart plugs (Tasmota, Shelly) and smart meters	Real-time data, no manual upload
NILM	Non-intrusive load monitoring to disaggregate appliances from a single meter	Appliance insight without per-device sensors
Smart device control	Automate schedules via Home Assistant / smart plug APIs	Turns recommendations into automatic actions
Reinforcement learning scheduler	RL agent optimizes appliance timing under TOU tariffs and comfort	Maximizes cost savings
Collaborative filtering	Once many households exist, matrix factorization for recommendations	Better personalization
LLM energy assistant	Chat interface answering "Why was my bill high?" grounded in the household's data	Accessible insights
Solar + battery optimization	Forecast PV generation and optimize self-consumption	Prosumer households
EV charging planner	Schedule EV charging for lowest cost/carbon	Growing load category
Gamification & community	Leaderboards, challenges, badges across apartments/hostels	Engagement
Utility portal	Aggregated anonymized demand-response insights	B2B revenue path
Mobile app	React Native app with push notifications	Daily engagement

Carbon-aware recommendations	Use real-time grid carbon intensity APIs	Lower emissions, not just kWh
------------------------------	--	-------------------------------